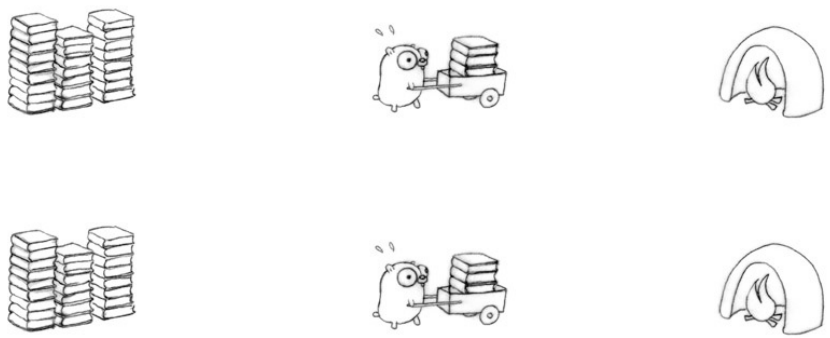
Concurrency

- In a single processor, programs executed "simultaneously"
- **Scheduler** distributes "processor time" to running programs (**processes**)
- Processes compete for processor access
- This is true **regardless** of the number of physical processors.

Parallelism vs Concurrency

Parallelism: Programming as the simultaneous execution of (possibly related) computations.

Concurrency: Programming as the composition of independently executing processes.

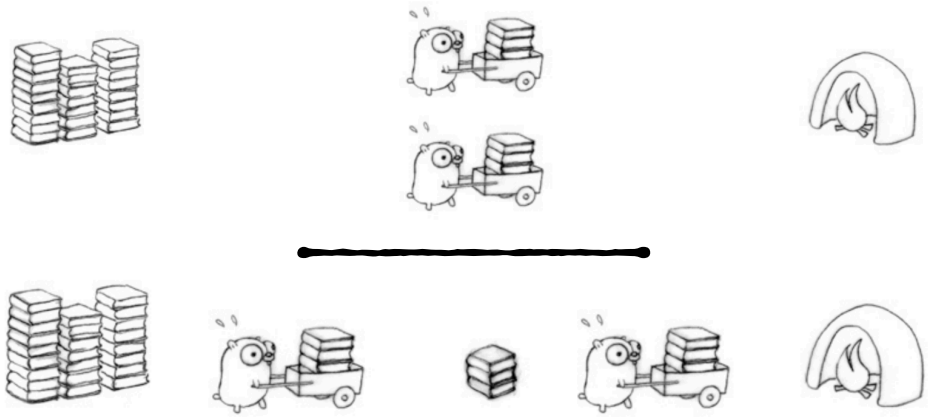


picture from https://talks.golang.org/

Parallelism vs Concurrency

Parallelism: Programming as the simultaneous execution of (possibly related) computations.

Concurrency: Programming as the composition of independently executing processes.



Concurrency vs Parallelism

- Concurrency is not parallelism, but parallelism is enabled by concurrency!
- Programs can be concurrent and have 0 parallelism.
- Well-written concurrency may run better on a multiprocessor.

Concurrency and Independence

- Concurrency is a way to **structure** work into independent pieces …

… but then you have to coordinate those pieces

Andrew Gerrand (Golang)

- "*Independent*" here refers to a way of *thinking* about problems, and *structuring* their solutions.

- Concurrent processes may indeed *interfere/interact*

Type of Concurrency

Shared memory concurrency:

- Processes coordinate by reading and writing to memory locations that are shared.

- Concurrent memory accesses managed by locks.

Message-passing concurrency:

- Processes coordinate by sending and receiving messages along **channels**.

- More abstract / higher-level than shared memory concurrency

- Channel operations need not be managed (processes may concurrently perform channel ops **safely**).

Challenges

- Reasoning about possible executions is a combinatoric problem — *state explosion problem*.

- With $P = P_1 ; P_2$ and $Q = Q_1 ; Q_2$, P and Q in parallel have 6 possible executions / interleavings.

- Errors can be quite difficult to diagnose / reproduce.

Safe Concurrency

What do we mean by safe? What can go wrong?

- Uncontrolled concurrent accesses to data **(Data Races)**

- Being stuck forever acquiring a lock **(Deadlock)**

- Being stuck forever trying to read / write to a channel **(Deadlock)**

- Repeating the same interaction without doing any useful work **(Livelock)**

Safe Concurrency

Correctness properties:

- **Mutual exclusion** (no conflicts — ensures safety/ consistency)
- **Deadlock-freedom** (the system as a whole is never fully “stuck”)
- **Progress** (No subsystem is stuck waiting forever)